

Sentry dla developerów

Program jest propozycją wstępną. Dla grupy 3 osób rekomendowany jest wariant 1-dniowy. Zakres można jednak elastycznie zmienić: wybrane moduły z programu podstawowego mogą zostać skrócone lub zastąpione modułami rozszerzonymi, jeśli klient chce położyć większy nacisk np. na **CI/CD, tracing, OpenTelemetry, dashboardy** albo **pracę z agentami AI**.

Dzień 1 - Sentry w praktyce developera: od konfiguracji do diagnozy błędu

Szacowany czas trwania: 6-7 godzin z przerwami

1.1 Sentry w workflow utrzymania aplikacji | ok. 45 min.

Cel: uczestnicy rozumieją, jak Sentry wspiera pracę developera od pierwszego alertu do decyzji o naprawie.

- Rola Sentry w monitoringu aplikacji i **developer observability**
- Issues, events, projects, environments, releases, tags, breadcrumbs i user context
- Jakie pytania powinno pomagać rozwiązać Sentry: co się stało, komu, od kiedy, w której wersji i z jakim wpływem
- Typowe problemy zespołów: brak środowisk, brak release, brak source maps, zbyt szerokie alerty, zbyt mało kontekstu
- **Ćwiczenie:** analiza przykładowego issue i wskazanie, jakich informacji brakuje do szybkiej diagnozy

1.2 Projekty, środowiska i konfiguracja SDK | ok. 1 godz.

Cel: uczestnicy potrafią zaprojektować bazową konfigurację Sentry dla aplikacji produkcyjnej.

- Organizacja, projekty, ownership i podział aplikacji na projekty
- Środowiska: production, staging, development, preview i zasady filtrowania danych
- **DSN**, release, environment, sample rate, tags i context
- Odfiltrowanie lokalnego developmentu, dev/stage oraz opcjonalnie własnego ruchu po IP
- Inbound filters, ignoreErrors, denyUrls/allowUrls, beforeSend i podstawowe **data scrubbing**
- **Ćwiczenie:** przygotowanie checklisty konfiguracji startowej dla projektu Sentry

1.3 Triage issues i analiza błędów | ok. 1 godz.

Cel: uczestnicy potrafią przejść od eventu w Sentry do hipotezy przyczyny i decyzji o priorytecie.

- **Stack trace**, in-app frames, breadcrumbs, request data, tags i user context
- Grouping, fingerprinting, priorytety, assignment i ownerzy
- Suspect commits, integracje z repozytorium i powiązanie błędu z kodem
- Ocena wpływu: liczba użytkowników, częstotliwość, regresja po release, trend w czasie

- Dobre praktyki komentowania, zamykania, ignorowania i eskalowania issues
- **Ćwiczenie:** triage kilku zgłoszeń i decyzja, które wymagają natychmiastowej reakcji

1.4 Releases, source maps i CI/CD | ok. 1 godz.

Cel: uczestnicy rozumieją, jak połączyć błędy z wersją aplikacji i uzyskać czytelne stack trace'y w produkcji.

- Release jako oś diagnostyczna: od kiedy błąd występuje i który deploy mógł go wprowadzić
- **Source maps** w aplikacjach JavaScript/TypeScript, Next.js/React (opcjonalnie Electron.js, Rust, Swift, itp.)
- Upload artefaktów w **CI/CD**, np. GitHub Actions
- Typowe problemy: upload po czasie, zły release name, brak debug IDs, niezgodne ścieżki
- Krótkie odniesienie do debug files i mapowania kodu poza JS, np. aplikacje natywne i desktopowe
- **Ćwiczenie:** przejście przez pipeline release + source maps i wskazanie krytycznych punktów konfiguracji

1.5 Tracing, logi i replay jako kontekst diagnozy | ok. 1 godz.

Cel: uczestnicy potrafią użyć danych kontekstowych do analizy problemów trudniejszych niż pojedynczy exception.

- **Performance monitoring**, transactions, spans i trace propagation
- Tracing frontend-backend i interpretacja waterfall view
- Logi powiązane z issue i trace
- **Session Replay** jako pomoc w odtworzeniu problemu użytkownika
- Sampling i koszty danych: co zbierać zawsze, co tylko próbkować
- **Ćwiczenie:** analiza scenariusza „użytkownik widzi błąd, backend odpowiada wolno” od issue do trace i logów

1.6 Monitory, alerty i redukcja szumu | ok. 1 godz.

Cel: uczestnicy potrafią zaprojektować alerting, który wspiera reakcję zespołu bez alert fatigue.

- Różnica między wykrywaniem problemu a routingiem powiadomień
- Alerty dla nowych issues, regresji, wzrostu błędów, wydajności, cron i uptime
- Slack, Teams, e-mail, Jira i PagerDuty jako kanały reakcji
- Progi, throttling, ownership, priorytety i eskalacja
- Praktyki ograniczania szumu: environment, issue type, filtry, sampling, reguły ignorowania
- **Ćwiczenie:** projekt zestawu alertów dla aplikacji produkcyjnej

1.7 AI w Sentry i workflow z agentami | ok. 45 min.

Cel: uczestnicy wiedzą, kiedy AI może przyspieszyć analizę i konfigurację, a kiedy lepiej pracować ręcznie.

- **Seer:** Root Cause Analysis, rekomendacje napraw, PR generation i AI Code Review
- **Sentry MCP** jako sposób udostępnienia agentowi danych o issues, traces i kontekście projektu
- Praca z GitHub Copilot, OpenAI Codex, Claude, Cursor, Zed lub JetBrains Junie przy konfiguracji, monitoringu i debugowaniu Sentry
- Przykładowe prompty do analizy błędu, przygotowania planu naprawy i audytu konfiguracji
- Automatyczne briefingi z najważniejszymi błędami i rekomendacjami
- Bezpieczeństwo: tokeny, uprawnienia, dane wrażliwe, kod Źródłowy i granice automatyzacji
- **Ćwiczenie:** przygotowanie promptu dla agenta AI na podstawie issue z Sentry

1.8 Podsumowanie i checklista wdrożeniowa | ok. 30 min.

Cel: uczestnicy wychodzą z konkretną listą działań do zastosowania w swoich projektach.

- Minimalny standard konfiguracji Sentry dla nowych projektów
- Checklista Środowisk, release, source maps, filtrów, alertów i ownership
- Standard pracy z incydem: od alertu do diagnozy i PR
- Decyzje do podjęcia po szkoleniu: projekty, uprawnienia, integracje, dashboardy, AI
- **Ćwiczenie:** mini-audyt przykładowej konfiguracji Sentry i wskazanie 5 najważniejszych zmian

Moduły opcjonalne - rozszerzenie do 2 dni

Poniższe moduły mogą zostać dodane jako drugi dzień szkolenia albo użyte do podmiany części programu podstawowego. Dzięki temu klient może zbudować wariant najlepiej dopasowany do tego, jak zespół faktycznie pracuje: **bardziej wdrożeniowo, bardziej operacyjnie, bardziej DevOpsowo** albo z **większym udziałem agentów AI**.

Moduł dodatkowy A - Sentry w CI/CD i release engineering | ok. 2-3 godz.

Cel: uczestnicy potrafią skonfigurować release workflow tak, aby każdy błąd był powiązany z wersją kodu i artefaktami potrzebnymi do debugowania.

- GitHub Actions lub inny pipeline CI/CD
- Release naming, deploy tracking i suspect commits
- Source maps, debug IDs, debug files i walidacja uploadu
- Troubleshooting source maps i najczęstsze problemy w pipeline
- **Ćwiczenie:** audyt przykładowego workflow CI/CD pod kątem Sentry

Moduł dodatkowy B - Distributed tracing i OpenTelemetry | ok. 2-3 godz.

Cel: uczestnicy rozumieją, jak diagnozować problemy rozproszone między frontendem, backendem i usługami zależnymi.

- **Trace propagation** między frontendem i backendem
- Transactions, spans, sampling i interpretacja trace
- Sentry SDK, **OpenTelemetry** i scenariusze hybrydowe
- OTLP, collector i integracje z istniejącym stackiem observability
- **Ćwiczenie:** diagnoza problemu w request flow na podstawie trace i logów

Moduł dodatkowy C - Dashboardy, metryki i operacyjny monitoring aplikacji | ok. 2 godz.

Cel: uczestnicy potrafią przygotować widoki, które pomagają zespołowi śledzić stan aplikacji bez ręcznego przeglądania pojedynczych issues.

- **Dashboardy** dla developerów, leadów technicznych i zespołów utrzymaniowych
- Application Metrics, trendy błędów, performance i release health
- Monitory dla cron, uptime i anomalii
- Jak dobrać widoki do codziennego briefingu technicznego
- **Ćwiczenie:** projekt dashboardu dla zespołu utrzymującego aplikację

Moduł dodatkowy D - AI-assisted debugging z Sentry MCP | ok. 1,5-2 godz.

Cel: uczestnicy potrafią bezpiecznie wykorzystać agenta AI do analizy błędów, konfiguracji Sentry i przygotowania planu naprawy.

- Konfiguracja **Sentry MCP** w wybranym narzędziu
- Zakres uprawnień, tokeny i bezpieczny dostęp do danych
- Prompty do analizy issue, trace, release i source maps
- Agent jako wsparcie konfiguracji SDK i CI/CD
- Daily briefing z błędów, priorytetów i rekomendacji napraw
- **Ćwiczenie:** agent analizuje issue i przygotowuje plan dalszej diagnostyki

Moduł dodatkowy E - Sentry dla aplikacji desktopowych i nietypowych stacków | ok. 1,5-2 godz.

Cel: uczestnicy poznają podejście do Sentry poza klasyczną aplikacją webową.

- Electron.js: proces main, Node.js i React view
- Aplikacje desktopowe, binarne i CLI
- Debug files, mapowanie kodu i diagnostyka błędów binarnych

- Różnice między błędami frontendowymi, backendowymi i runtime desktop
- **Ćwiczenie:** zaprojektowanie konfiguracji Sentry dla aplikacji desktopowej lub CLI

Rekomendowany wariant realizacji

Dla grupy 3 osób rekomendowany jest 1 dzień jako wariant podstawowy. Pozwoli to przejść przez pełny cykl pracy: konfigurację, środowiska, triage, releases, source maps, podstawy tracingu, alerting i krótkie omówienie AI.

Wariant 2-dniowy warto wybrać, jeśli klient chce głębiej przećwiczyć CI/CD, distributed tracing, OpenTelemetry, dashboardy, pracę z agentami AI albo nietypowe stacki. Drugi dzień może być zbudowany modułowo: nie trzeba realizować wszystkich modułów opcjonalnych, można wybrać te, które odpowiadają realnemu środowisku uczestników.

Możliwy jest też wariant mieszany: część modułów podstawowych skracamy, a w ich miejsce dodajemy wybrany moduł rozszerzony. To dobre rozwiązanie, jeśli uczestnicy mają już doświadczenie z podstawami Sentry i potrzebują głębszej pracy w konkretnym obszarze.